DDQN Double Deep Q-Network

DDPG Deep Deterministic Policy Gradient

MoE Mixture of Experts

SAC Soft Actor Critic

SARSA State-action-reward-state-action

TD Temporal Difference

TD3 Twin Delayed Deep Deterministic Policy Gradient

RL-Course 2024/25: Final Project Report

Reinforce the Puck: Jonathan Schwab, Tom Freudenmann

February 18, 2025

1 Introduction

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetuer.

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.
Sed commodo posuere pede. Mauris ut est. Ut quis purus. Sed ac odio. Sed vehicula hendrerit sem. Duis non odio. Morbi ut dui. Sed accumsan risus eget odio. In hac habitasse platea dictumst. Pellentesque non elit. Fusce sed justo eu urna porta tincidunt. Mauris felis odio, sollicitudin sed, volutpat a, ornare ac, erat. Morbi quis dolor. Donec pellentesque, erat ac sagittis semper, nunc dui lobortis purus, quis congue purus metus ultricies tellus. Proin et quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent sapien turpis, fermentum vel, eleifend faucibus, vehicula eu, lacus.

2 Method

2.1 Twin Delayed Deep Deterministic Policy Gradient

Our Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm is a from scratch implementation of the original paper [1]. TD3 is an off-policy actor-critic algorithm for continous action and state spaces. This combination is very vulnerable to overestimation bias and error accumulation in the Temporal Difference (TD)-learning [1]. The overestimation bias is related to discrete Q-learning, where the value estimate is updated with the following greedy target:

$$y = r + \gamma \max_{a'} Q(s', a') \leq \mathbb{E}_{\epsilon} \left[max_{a'}(Q(s', a') + \epsilon) \right] \tag{1}$$

where r is the reward, γ the discount factor, s' the next state and a' the next action. By estimating with error ϵ the result is generally greater than the true maximum [7]. This problem can also be proofed for actor-critic environments, where the policy is updated with the deterministic policy gradient [1] and leads to suboptimal updates and diverging behavior in the learning process.
Therefore, TD3 introduces the clipped double Q-Learning, which is based on Double Deep Q-Network (DDQN). DDQN optimizes two actors with two critics, which use the same replay buffer and the opposite critic in the learning process [9], inducing an value overestimation of some states. In difference TD3 uses a single actor π_ϕ which is optimized with respect to the minimum of two critics $Q_{\theta_1}, Q_{\theta_2}$:

$$y = r + \gamma \min_{i=1,2} Q_{\theta'_i}(s', \pi_\phi(s')) \tag{2}$$

As result, the computational costs are reduced by using a single actor and an underestimation bias is induced which is preferable to overestimation since it does not propagate through the policy updates [1]. Another problem are the high variance estimates causing a noisy policy gradient. Thus, the Bellman equation is never exactly satisfied, which results in a small residual TD-error δ_t. This error propagates through the update step, such that the expected return depends on the expected discounted difference between reward and TD-error [1]:

$$Q_\theta(s_t, a_t) = \mathbb{E}_{s_i \sim p_\pi, a_i \sim \pi} \left[\sum_{i=t}^{T} \gamma^{i-t}(r_i - \delta_i) \right] \tag{3}$$

TD3 addresses this problem by using slowly changing target networks for actor and critics and introducing policy smoothing regularization. Firstly, target networks are used to reduce the error over multiple updates by providing a stable learning objective and reducing the variance in the policy updates [1]. In TD3 the target networks parameters θ are updated with a small rate τ:

$$\theta' \leftarrow \tau\theta + (1-\tau)\theta' \tag{4}$$

Secondly, target policy smoothing regularization is introduced to reduce the vulnerability to narrow peaks in the values estimate. The main idea is that similar actions in the continous space should have similar values, which is related to the learning update from State-action-reward-state-action (SARSA) algorithm [6]. To enforce the relationship between similar actions, a small area around the target action is fitted by adding a small amount of Gaussian clipped noise. This results in the following update step:

$$y = r + \gamma Q_{\theta'}(s', \pi_{\phi'}(s') + \epsilon) \quad |\epsilon \sim clip(\mathcal{N}(0,\sigma), -c, c) \tag{5}$$

Moreover, TD3 extends the basic Deep Deterministic Policy Gradient (DDPG) algorithm [5] by resolving the overestimation bias, adding soft target network updates and smoothing the target policy to lower the further the variance in the value estimate. As result, it is a state of the art actor-critic algorithm, which shows an improvement in learning speed and performance compared to DDPG in some tasks in continuous domains .

2.2 Soft Actor Critic

Our Soft Actor Critic (SAC) implementation is based on the original paper by [3] and its follow-up paper [4]. SAC is an off-policy actor-critic algorithm that learns a stochastic policy in an environment with continuous action and state spaces. The algorithm is based on the principle of maximum entropy reinforcement learning, which aims to maximize the expected reward while also maximizing the entropy of the policy. This encourages exploration and prevents the policy from getting stuck in local optima. The major components of the SAC algorithm are the actor, the critic, and the entropy target:

$$J(\pi) = \sum_{t=0}^{T} \mathbb{E}_{(s_t,a_t)\sim\rho_\pi}\left[r(s_t,a_t) + \alpha\mathcal{H}(\pi(\cdot \mid s_t))\right] \tag{6}$$

The actor is a stochastic policy network, which is based on a vanilla neural network with a Gaussian distribution output layer. It estimates the mean and standard deviation $\mu(s), \sigma(s) = f_\theta(s)$ and provides a sampling method which returns both a sample from the distribution and the log probability of the selected action. To make the sampling process differentiable, the reparameterization trick is used where $z \sim \mathcal{N}(0,1)$ and $u = \mu(s)+\sigma(s)\cdot z$. The action bounds are enforced by applying the $\tanh$ function to the sampled unbounded action. To consider this bounding in the probability density function, the probability density function of the unbounded action is scaled by the absolute determinant of the Jacobian matrix of the $\tanh$ function [1][C. Enforcing Action Bounds]:

$$\pi(a \mid s) = \mu(u \mid s)\left|\det\left(\frac{d\mathbf{a}}{d\mathbf{u}}\right)\right|^{-1}. \tag{7}$$

So for each state, sampled from the replay buffer $\mathcal{D}$ an action is sampled from the actor and the log probability of this action is calculated using the adjusted probability density function. The actor is trained using the maximum entropy objective, where the critic estimates the value for each state-action pair:

$$J(\pi) = \mathbb{E}_{s_t\sim\mathcal{D}, a_t\sim\pi}\left[\alpha \log \pi(a_t \mid s_t) - Q(s_t,a_t)\right] \tag{8}$$

The critic consists of two Q-networks, valuing state-action pairs, to prevent overestimation bias [2]. During training both networks independently predict a value for the given state-action pair $(s_t \sim \mathcal{D}, a_t \sim \pi)$ and the minimum is selected.
The target Q-values incorporate the immediate reward r and the discounted future reward. Additionally, an entropy term, weighted by the temperature parameter α, is used to balance exploration and exploitation. This formulation aligns with the objective function $J(\pi)$, which maximizes both the expected reward and the entropy of the policy:

$$y = r + \gamma(1-d)\left(\min\left(Q_1^{\text{target}}(s', a'), Q_2^{\text{target}}(s', a')\right) + \alpha\mathcal{H}(\pi(\cdot \mid s'))\right). \tag{9}$$

The critic is trained by minimizing the mean squared Bellman error, which represents the difference between the estimated Q-values and the target values:

$$\mathcal{L}_Q = \mathbb{E}\left[(Q(s,a) - y)^2\right]. \tag{10}$$

To stabilize the training process, the target Q-networks are updated using a soft target update mechanism, as it was shown to improve the stability of the training process [3][E. Additional Baseline Results].
In the original paper “an additional function approximator for the value function [was introduced] but later found [as] to be unnecessary” [4]

Moreover the automatically temperature parameter tuning using gradient-based optimization, introduced in the follow-up paper [4], was implemented:

$$J(\alpha) = \mathbb{E}a_t \sim \pi\left[-\alpha\left(\log\pi(a_t \mid s_t) + \mathcal{H}\text{target}\right)\right], \tag{11}$$

where $\mathcal{H}$target is set to negative of the action space dimensionality, as proposed in [4][D. Hyperparameters]

2.3 Hybrid-Model

While reviewing the gameplay of the trained SAC and TD3 agents, we noticed that both agents have their strengths and weaknesses in different situations. To combine the strengths of both agents, we created a hybrid model. This approach is inspired by the Mixture of Experts (MoE) architecture, where multiple experts are combined using a gating network to weight the experts' predictions. But since we've got an continious spatial action space, weighting both 'experts' predictions would not be feasible option. Thus a DDQN [8] was implemented as selection gate for the actions. This gate values the predicted actions of both agents and select the best action for each state:

$$a_{\text{Hybrid}} = \arg\max_{a \in \{\pi_{\text{SAC}}(s), \pi_{\text{TD3}}(s)\}} Q_{\text{DDQN}}(s,a) \tag{12}$$

The loss function for training the DDQN is based on the TD-Error, which is the difference between the predicted and the target Q-value. As a major difference to the MoE approach the 'expert' models are fixed (trained independently) and not trained simulataneously to the gating network.

References

[1] S. Fujimoto, H. van Hoof, and D. Meger. Addressing function approximation error in actor-critic methods. In J. Dy and A. Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1587–1596, Stockholmsmässan, Stockholm Sweden, 10–15 Jul 2018. PMLR.

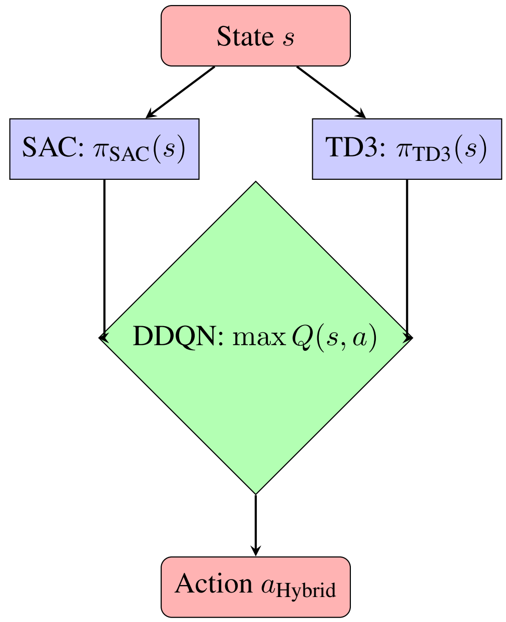

Figure 1: Hybrid-Model with DDQN Selection Gate

[2] S. Fujimoto, H. van Hoof, and D. Meger. Addressing function approximation error in actor-critic methods, 2018.

[3] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In J. Dy and A. Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1861–1870, Stockholmsmässan, Stockholm Sweden, 10–15 Jul 2018. PMLR.

[4] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine. Soft actor-critic algorithms and applications. 2019.

[5] T. Lillicrap. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

[6] R. S. Sutton and A. G. Barto. Reinforcement learning: An introduction. *Robotica*, 17(2):229–235, 1999.

[7] S. Thrun and A. Schwartz. Issues in using function approximation for reinforcement learning. In *Proceedings of the 1993 connectionist models summer school*, pages 255–263. Psychology Press, 2014.

[8] H. van Hasselt, A. Guez, and D. Silver. Deep reinforcement learning with double q-learning, 2015.

[9] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas. Dueling network architectures for deep reinforcement learning. In M. F. Balcan and K. Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1995–2003, New York, New York, USA, 20–22 Jun 2016. PMLR.